

Akamata ハンドブック — 15 分で始めるガイド

Akamata は Zig 0.16 製の Web フレームワークです。ひとつのソースコードから 2 通りのデプロイ形態を生成できます: native バイナリ (VPS / Cloudflare Containers) と Cloudflare Workers の wasm モジュール。DB 層は SQLite、Turso (libsql)、Cloudflare D1 を URL の違いだけで透過的に切り替えます — ハンドラのコードはどのバックエンドで動いているかを意識しません。

このドキュメントは CRUD API を 1 本書き上げるのに必要な情報を網羅します。各章は約 2 分で読めます。必要なところだけ拾い読みしてください。

0. インストール

```
git clone https://github.com/yourorg/Akamata
cd Akamata
zig build cli      # ./zig-out/bin/akamata
# (任意) zig-out/bin を PATH に追加すれば任意のディレクトリから `akamata` を呼べる
```

以下も併せて入れておくと便利です:

- `node` + `wrangler` (Workers デプロイ、ローカルの `wrangler dev` に必要)
- `turso` CLI (Turso を使う場合のみ)
- `docker` (Cloudflare Containers にデプロイする場合のみ)

1. プロジェクトの雛形生成 (30 秒)

```
akamata init mynotes --target=both
cd mynotes
zig build run
# → mynotes listening on :8080
```

`--target=both` を指定すると、native (`src/main.zig`) と Workers (`src/worker.zig`) の両方のエントリーポイントが生成されます。生成された `main.zig` は 1 ファイルで現代的なモデルベースのワークフローを丸ごとデモしています — `Note` モデル、バリデーション、スキーママイグレーション、CRUD ハンドラがすでに揃っています。

動作確認:

```
curl -sS -X POST -H 'content-type: application/json' \
  -d '{"title":"hi","body":"first note"}' \
  http://127.0.0.1:8080/notes
curl -sS http://127.0.0.1:8080/notes
```

2. モデル (3 分)

モデルは `pub const __schema` ブロックを持った普通の Zig struct です。Akamata がコンパイル時に内省して、CREATE TABLE SQL、バリデータ、クエリ、マイグレーションを自動生成します。

```
pub const User = struct {
    id: ?i64 = null,                // ?i64 = null → INTEGER PRIMARY KEY AUTOINCREMENT
    email: []const u8,              // TEXT NOT NULL
    name: []const u8,
    age: ?i32 = null,               // INTEGER (NULL 許容)
    created_at: ?i64 = null,        // INTEGER DEFAULT (unixepoch())

    pub const __schema = .{
        .table = "users",           // 任意。既定値は struct 名 + "s" の小文字
        .primary_key = "id",        // 任意。既定値は "id"
        .indexes = .{
            .{ "email", .unique },   // 単一カラムのユニークインデックス → email_unq
            .{ "name", .index },     // 非ユニーク
        },
        .defaults = .{
            .created_at = "unixepoch()",
        },
        .validates = .{
            .email = .{ am.model.rule.required, am.model.rule.max_len(255), am.model.rule.fc
            .name = .{ am.model.rule.required, am.model.rule.max_len(80) },
            .age = .{ am.model.rule.range(0, 150) },
        },
        // 任意: Zig フィールド名 → SQL カラム名のリネーム
        // .columns = .{ .userId = "user_id" },
        // 任意: リレーション
        // .relations = .{
        //     .posts = .{ .has_many = .{ .model = Post, .fk = "user_id" } },
        // },
    };
};
```

カスタムバリデータ

```
fn requireAcmeDomain(value: []const u8, _: std.mem.Allocator) ?[]const u8 {
    return if (std.mem.endsWith(u8, value, "@acme.co")) null else "acme.co のメールアドレスを指
}

pub const __schema = .{
    .validates = .{
        .email = .{ am.model.rule.required, am.model.rule.custom(requireAcmeDomain) },
    },
};
```

戻り値が `null` なら OK、文字列を返したらそれがエラーメッセージになります。整数フィールド向けには `am.model.rule.customInt(fn)` も用意されています。

3. Repo: 型安全な CRUD (2 分)

```
const Users = am.model.repo(User);

// 読み取り
const u = try Users.find(c.db(), c.arena, 42);           // ?User
const all = try Users.all(c.db(), c.arena);              // []User (新しい順)
const adults = try Users.where(c.db(), c.arena, .{ .age = 30 });

// 書き込み
const created = try Users.create(c.db(), c.arena, .{ .email = "x@y", .name = "x" });
var u2 = created;
u2.name = "renamed";
try Users.save(c.db(), c.arena, &u2);
try Users.delete(c.db(), u2.id.?);

// 抜け道 - 任意の SQL を投げつつ結果は User にマップ
const old = try Users.queryRaw(c.db(), c.arena,
    "SELECT id, email, name, age, created_at FROM users WHERE age > ? ORDER BY age DESC LIMIT {30}",
    .{30},
);

// Eager loading (N+1 撲滅)
const owners = try Users.all(c.db(), c.arena);
const loaded = try am.model.preload.hasMany(User, "posts", owners, c.db(), c.arena);
for (loaded) |row| {
    // row.parent: User; row.related: []Post (1 回の IN クエリで取得)
}
```

4. ハンドラ: 最短記述版 (2 分)

```
const Ctx = am.Context(State);

fn createNote(c: *Ctx) !void {
    // JSON パース + __schema.validates 実行。JSON が不正なら 400 を返して null。
    // バリデーション失敗なら 422 を返して null。どちらにせよハンドラは早期 return。
    const note = (try c.input(Note)) orelse return;
    const created = try Notes.create(c.db(), c.arena, note);
    try c.json(created, 201);
}

fn showNote(c: *Ctx) !void {
    const id = c.req.paramAs(i64, "id") catch return c.badRequest("invalid id");
    const n = (try Notes.find(c.db(), c.arena, id)) orelse return c.notFound();
    try c.json(n, 200);
}
```

エラーレスポンス短縮形 (すべて { error_kind, message? } の一貫した JSON を返す)

メソッド	ステータス	用途
<code>c.badRequest(msg)</code>	400	リクエスト形式不正
<code>c.unauthorized(msg)</code>	401	認証情報なし / 不正
<code>c.forbidden(msg)</code>	403	認証済みだが権限なし
<code>c.notFound()</code>	404	リソースが存在しない
<code>c.conflict(msg)</code>	409	重複・状態不整合
<code>c.unprocessable(errs)</code>	422	バリデーション失敗
<code>c.serverError(msg)</code>	500	汎用サーバエラー
<code>c.json(value, code)</code>	任意	カスタムコード

State / config へのショートカット

```
pub const State = struct {
    db: am.db.Db,
    cfg: MyConfig,
};

fn handler(c: *Ctx) !void {
    _ = c.db(); // c.state().db と同じ
    _ = c.cfg(); // c.state().cfg と同じ
}
```

5. ルーティング + ミドルウェア (1 分)

```
pub fn registerRoutes(app: *am.App(State)) !void {
    _ = try app.useAll(am.mw.recover(State));
    _ = try app.useAll(am.mw.logger(State));
    _ = try app.useAll(am.mw.requestId(State)); // X-Request-ID を付与
    _ = try app.useAll(am.mw.accessLog(State, .json)); // 構造化アクセスログ
    _ = try app.useAll(am.mw.cors(State, .{})); // CORS プリフライト
    _ = try app.useAll(am.mw.rateLimit(State, .{ .max = 100, .per_secs = 60 }));

    _ = try app.get("/notes", listNotes);
    _ = try app.post("/notes", createNote);
    _ = try app.get("/notes/:id", showNote);
    _ = try app.delete("/notes/:id", deleteNote);

    _ = try app.ws("/live", liveHandler);
}
```

6. DATABASE_URL でバックエンドを選択 (1 分)

URL スキーム	バックエンド	動作環境
<code>file:./mynotes.db</code>	SQLite	native のみ
<code>libsql://<db>.turso.io?authToken=...</code>	Turso (libsql HTTP)	native + Workers
<code>https://<db>.turso.io?authToken=...</code>	Turso (HTTPS エイリアス)	native + Workers
<code>d1:DB</code>	Cloudflare D1 binding	Workers のみ

ローカル SQLite (既定値)

```
zig build run
```

Turso

```
DATABASE_URL='libsql://my.turso.io?authToken=eyJ...' zig build run
```

Workers の場合は `deploy/wrangler.toml` の `[vars]` ブロックで `DATABASE_URL` を設定します。

7. マイグレーション (2 分)

2 種類のスタイルがあります。プロジェクトの段階に応じて使い分けます。

A. 自動 diff (初期開発に向く)

雛形では native 起動時に毎回これが走るようになっていきます:

```
const plan = try am.model.migrate.diff(arena, db, &all_models);
try am.model.migrate.apply(arena, db, plan);
```

Workers では `migrate_once.run` ミドルウェアを使います (雛形が最初の HTTP リクエスト時に走るよう自動で組み込みます。 `akamata_init` 時点では JSPI がまだ有効でないため、初回リクエストで実行するのがポイント)。

B. バージョン付きファイル (本番運用に向く)

```
akamata migrate generate add_users          # migrations/<ts>_add_users.sql を生成
# ...SQL を編集...
./zig-out/bin/myapp migrate-up              # 未適用分を実行、schema_migrations に記録
```

`migrate-up` は雛形 `main.zig` に組み込まれているサブコマンドで、内部で `am.model.migrate.Migrator` を呼びます。独自ロックや dry-run を入れたい 場合はその部分を書き換えてください。

8. デプロイ (3 分)

VPS / Container

```
zig build -Doptimize=ReleaseFast -Dtarget=x86_64-linux-musl
# zig-out/bin/myapp が musl 静的バイナリとして出力される
```

`deploy/Dockerfile` (akamata init で生成) は `FROM scratch + COPY` だけの 最小構成です。 `docker build -f deploy/Dockerfile -t myapp .` で完了。

Cloudflare Workers + D1

```
akamata deploy --workers \  
  --config=deploy/wrangler.toml \  
  --migrate=<(/zig-out/bin/myapp --print-schema)
```

このコマンド一発で:

1. `wrangler.toml` の `[[d1_databases]]` ブロックを読む
2. `database_id` がプレースホルダ (`00000000-...`) なら `wrangler d1 create <name>` を実行し、得た UUID を `wrangler.toml` に書き戻す。すでに同名 DB がアカウントに存在する場合は `wrangler d1 list --json` で UUID を取得して採用する
3. `--migrate` で渡されたスキーマを本番 D1 に適用する
4. `zig build -Dbackend=workers -Doptimize=ReleaseSmall` でビルド
5. `wrangler deploy --config=...` でデプロイ

`<(...)>` はプロセス置換で、テンポラリファイル無しでスキーマを直接パイプ できます。普通のファイルパスでも問題ありません: `--migrate=migrations/00_init.sql`。

Cloudflare Workers + Turso

D1 と同じ流れですが、`wrangler.toml` を次のように書き換えます:

```
[vars]  
DATABASE_URL = "libsql://<db>.turso.io?authToken=..."  
# [[d1_databases]] は削除する
```

`akamata deploy --workers --config=deploy/wrangler.toml` を実行すれば、wasm 側は JSPI を介して `fetch()` で libsql/Hrana プロトコルを話し、同一のハンドラコードがそのまま動きます。

9. チートシート

```
akamata init <name> [--target=native|workers|containers|both]
akamata build [--workers|--containers]
akamata dev
akamata deploy [--workers|--containers] [--config=PATH] [--migrate=SQL]
akamata db <sql-file> [--local|--remote] [--config=PATH]
akamata migrate generate <name> [--dir=migrations]
akamata migrate up [--dir=migrations]
akamata --version
akamata help
```

タイポした時:

```
$ akamata deplyo
akamata: unknown subcommand `deplyo`
Did you mean `akamata deploy`?
```

ハンドラ内で使える関数:

```
c.db() == c.state().db
c.cfg() == c.state().cfg
c.input(T) パース + バリデーション → ?T (失敗時は 400/422 を返す)
c.json(value, code)
c.badRequest / c.notFound / c.unauthorized / c.forbidden / c.conflict / c.unprocessable / c.
c.req.json(T) 生パース (バリデーションなし)
c.req.paramAs(i64, "id")
c.req.query("q")
c.req.header("x-foo")
c.req.cookie("session")
c.setCookie(name, value, .{ .secure = true })
```

モデル層:

```
am.model.tableDef(T) コンパイル時 TableDef
am.model.repo(T) find/all/where/create/save/delete/queryRaw を持つ Repo
am.model.validate(T, v, arena)
am.model.migrate.diff/apply
am.model.migrate.Once Workers 向け遅延実行ガード
am.model.migrate.Migrator バージョン付きファイルランナー
am.model.preload.hasMany(Owner, "rel", parents, db, arena)
am.model.relations.hasMany / belongsTo (遅延ロード)
```

さらに進むには

- [examples/guestbook/](#) — 必要最小限の完動アプリ。HTML UI、バリデーション、3つのDBバックエンドすべてに対応
- [examples/mobus/](#) — 規模のある実用アプリ (認証、フレンド、メッセージ、WebSocket ハブ、FCM push)
- [examples/chat/](#) — Durable Object SQLite + WebSocket
- [docs/db-backends.md](#) — バックエンド実装メモ (JSPI、Hrana プロトコル詳細)

- `docs/benchmarks.md` — 性能計測結果 (ホットパスで 167k req/s)
- `docs/architecture.md` — フレームワーク内部構造 + 本番運用向けハードニングのまとめ